

Foundation of Blockchain Technology (IS501) Pre-Reading Material – Lecture 14

Getting Started with Solidity

Estimated Reading Time: 15–20 minutes

Before the Lecture

Solidity is a programming language used to develop **smart contracts on Ethereum**. These contracts run on the **Ethereum Virtual Machine (EVM)**, where the same code is executed across different blockchain nodes. Therefore, Solidity programs should produce the same output for the same input.

A Solidity program is written as human-readable source code and then **compiled into bytecode**, which is executed by the EVM.

A simple Solidity contract looks like:

```
pragma solidity ^0.8.0;  
  
contract HelloWorld {  
    string public message = "Hello World";  
}
```

A Solidity file generally contains a **version declaration, comments, imports (if required), and a contract definition**. The contract body contains variables and functions.

Data in Solidity

Smart contracts work with different types of data. Two important categories of variables are:

- **Local variables** – temporary and used during function execution.
- **State variables** – stored permanently on the blockchain.

Solidity also provides basic data types such as:

- `uint` – non-negative numbers
- `int` – positive and negative numbers
- `bool` – true or false



The choice of data type and storage location can affect the efficiency and gas usage of a smart contract.

Visibility

Visibility modifiers control who can access variables and functions:

public | external | internal | private

Understanding these modifiers is important for controlling access to a smart contract.

Think Before the Lecture

1. What is the role of the EVM?
2. Why is a state variable different from a local variable?
3. What is the purpose of `public` and `private`?
4. Which data type would you use for a value that can be either `true` or `false`?

